

NAME: A.DILEEP KUMAR

Reg no: 192372074

Course : Theory of computations (CSA1303)

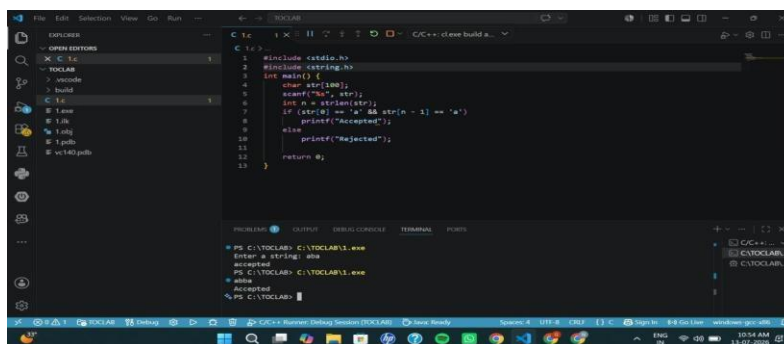
Exercise 1

Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    scanf("%s", str);
    int n = strlen(str);
    if (str[0] == 'a' && str[n - 1] == 'a')
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

Output:

A screenshot of a C++ IDE (Visual Studio Code) showing the code and its execution. The code is a C program that checks if a string starts and ends with 'a'. The output shows the program running and accepting the input string 'aba'.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char str[100];
6     scanf("%s", str);
7     int n = strlen(str);
8     if (str[0] == 'a' && str[n - 1] == 'a')
9         printf("Accepted");
10    else
11        printf("Rejected");
12    return 0;
13 }
```

Program output:

```
PS C:\TDC\LAB> C:\TDC\LAB\1.exe
Enter a string: aba
Accepted
PS C:\TDC\LAB> C:\TDC\LAB\1.exe
aba
Accepted
PS C:\TDC\LAB>
```

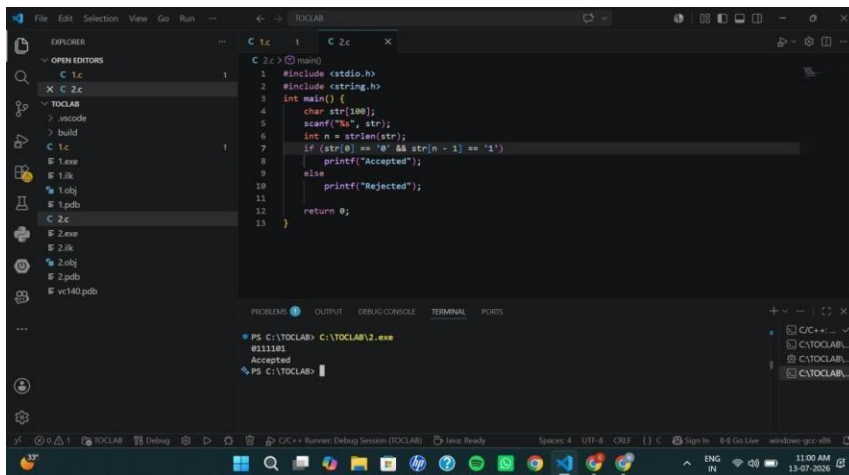
Exercise 2

Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    scanf("%s", str);
    int n = strlen(str);
    if (str[0] == '0' && str[n - 1] == '1')
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

Output:



Exercise 3

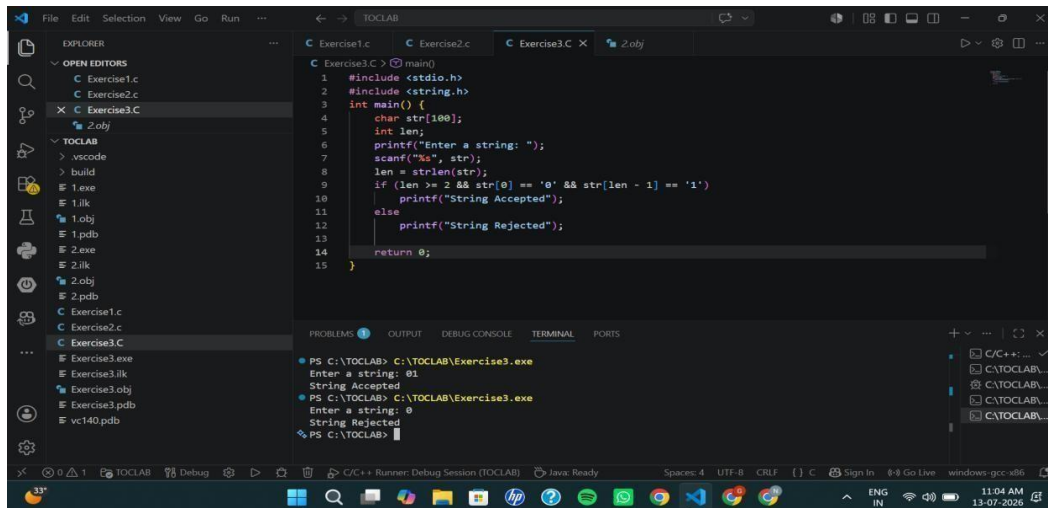
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0A1$ $A \rightarrow 0A \mid 1A \mid \epsilon$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int len;
    printf("Enter a string: ");
    scanf("%s", str);
    len = strlen(str);
    if (len >= 2 && str[0] == '0' && str[len - 1] == '1')
        printf("String Accepted");
    else
        printf("String Rejected");
    return 0;
}
```

Output:



```
File Edit Selection View Go Run ... TOCLAB
EXPLORER
  OPEN EDITORS
    Exercise1.c
    Exercise2.c
    Exercise3.c
  TOCLAB
    .vscode
    build
    1.exe
    1.ilk
    1.obj
    1.pdb
    2.exe
    2.ilk
    2.obj
    2.pdb
    Exercise1.c
    Exercise2.c
    Exercise3.c
    Exercise3.exe
    Exercise3.ilk
    Exercise3.obj
    Exercise3.pdb
    vc140.pdb
  ...
  Settings

C Exercise3.c Exercise2.c Exercise3.c X 2.obj
C Exercise3.c main()
1 #include <stdio.h>
2 #include <string.h>
3 int main() {
4     char str[100];
5     int len;
6     printf("Enter a string: ");
7     scanf("%s", str);
8     len = strlen(str);
9     if (len >= 2 && str[0] == '0' && str[len - 1] == '1')
10         printf("String Accepted");
11     else
12         printf("String Rejected");
13
14     return 0;
15 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\TOCLAB> C:\TOCLAB\Exercise3.exe
Enter a string: 01
String Accepted
PS C:\TOCLAB> C:\TOCLAB\Exercise3.exe
Enter a string: 0
String Rejected
PS C:\TOCLAB>

C/C++ Runner: Debug Session (TOCLAB) Java: Ready Spaces: 4 UTF-8 CRLF {} C Sign In B-$ Go Live windows-gcc-x86
11:04 AM 13-07-2026
```

Exercise 4

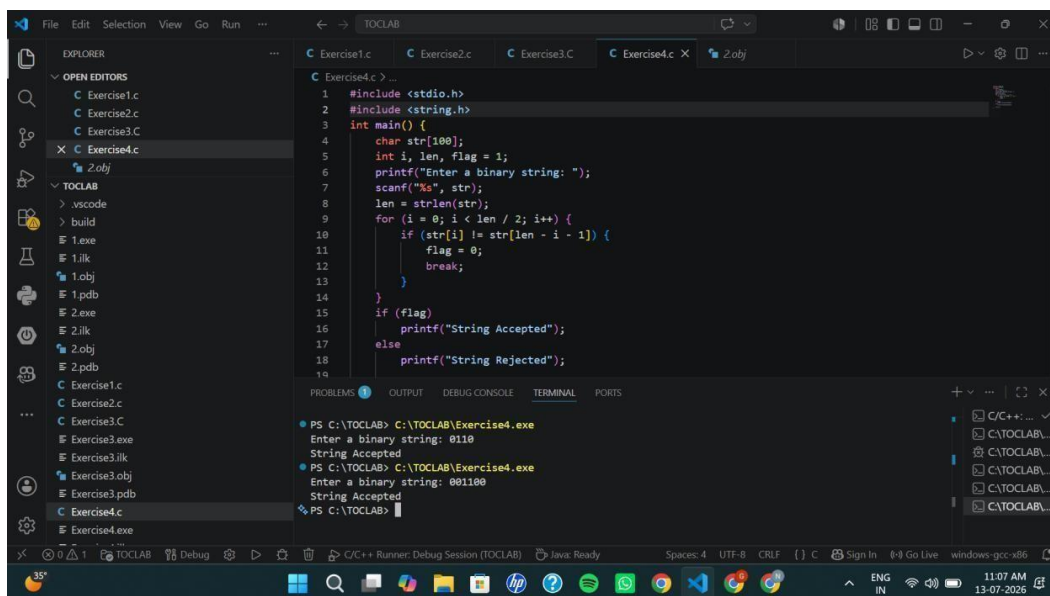
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, len, flag = 1;
    printf("Enter a binary string: ");
    scanf("%s", str);
    len = strlen(str);
    for (i = 0; i < len / 2; i++) {
        if (str[i] != str[len - i - 1]) {
            flag = 0;
            break;
        }
    }
    if (flag)
        printf("String Accepted");
    else
        printf("String Rejected");
    return 0;
}
```

Output:



Exercise 5

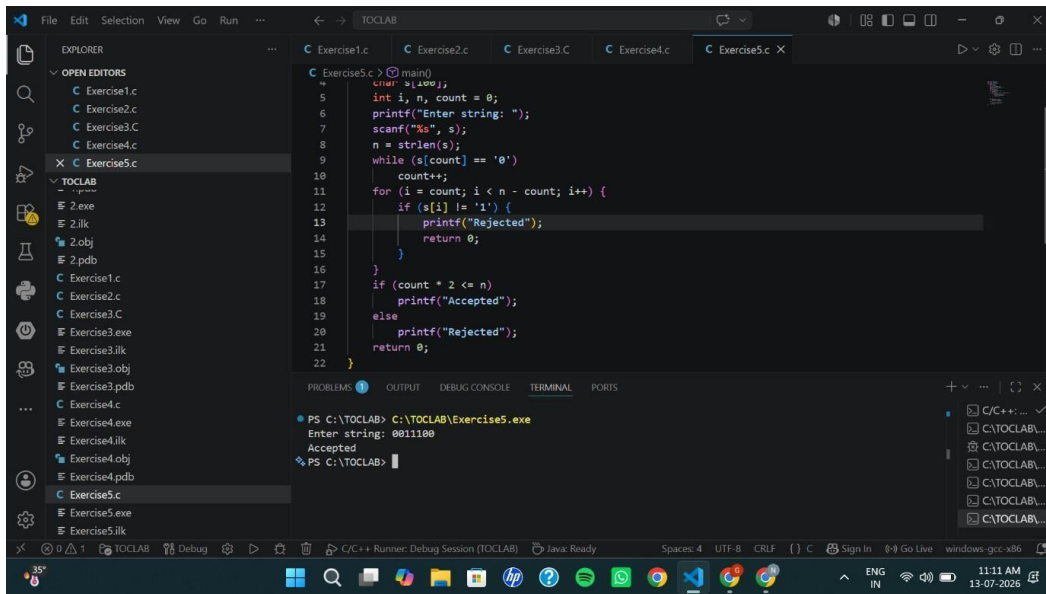
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid A \quad A \rightarrow 1A \mid \varepsilon$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i, n, count = 0;
    printf("Enter string: ");
    scanf("%s", s);
    n = strlen(s);
    while (s[count] == '0')
        count++;
    for (i = count; i < n - count; i++) {
        if (s[i] != '1') {
            printf("Rejected");
            return 0;
        }
    }
    if (count * 2 <= n)
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

Output:



Exercise 6

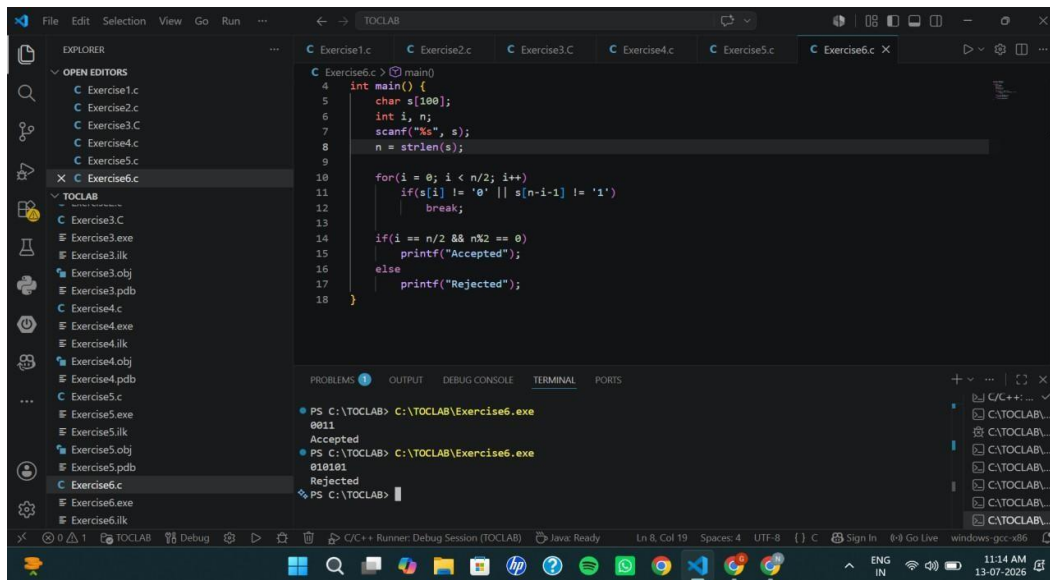
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S1 \mid \epsilon$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i, n;
    scanf("%s", s);
    n = strlen(s);
    for(i = 0; i < n/2; i++)
        if(s[i] != '0' || s[n-i-1] != '1')
            break;
    if(i == n/2 && n%2 == 0)
        printf("Accepted");
    else
        printf("Rejected");
}
```

Output:



Exercise 7

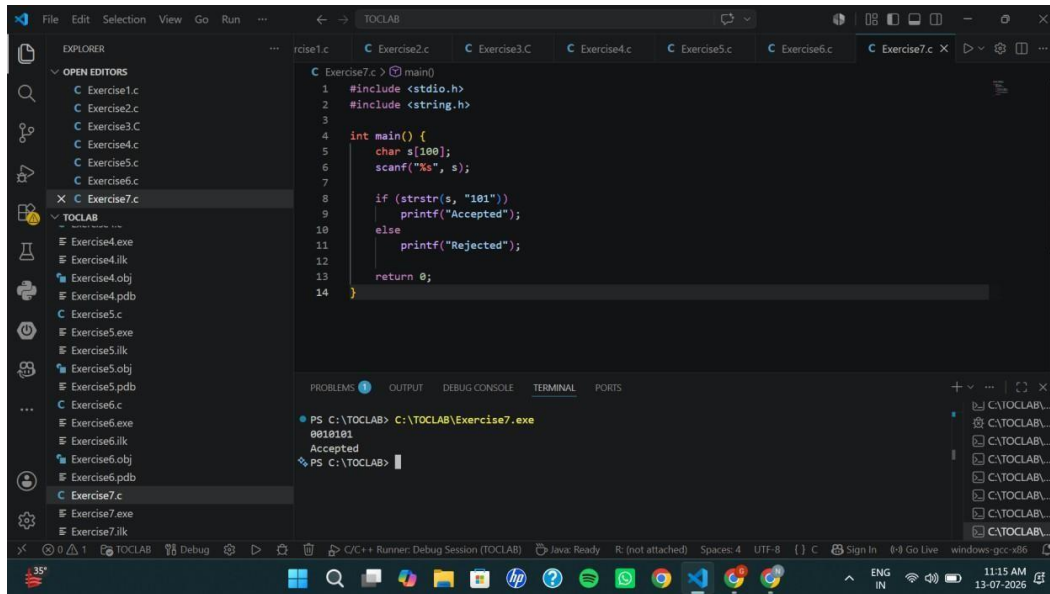
Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow A101A$, $A \rightarrow 0A \mid 1A \mid \varepsilon$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    scanf("%s", s);
    if (strstr(s, "101"))
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

Output:



Exercise 8:

Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len;

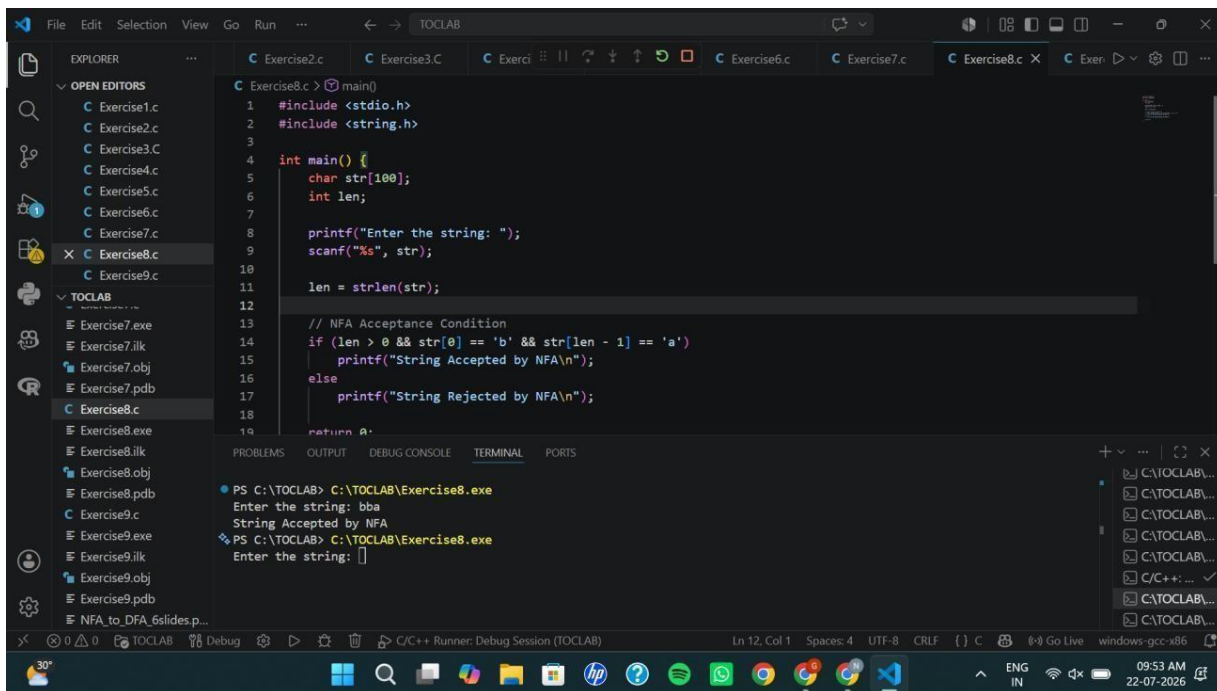
    printf("Enter the string: ");
    scanf("%s", str);

    len = strlen(str);

    // NFA Acceptance Condition
    if (len > 0 && str[0] == 'b' && str[len - 1] == 'a')
        printf("String Accepted by NFA\n");
    else
        printf("String Rejected by NFA\n");

    return 0;
}
```

Output:



The screenshot shows a Visual Studio Code editor window with the following components:

- EXPLORER:** A list of files on the left, including Exercise1.c through Exercise9.c, and TOCLAB files (Exercise7.exe, .ilk, .obj, .pdb, and Exercise8.c).
- EDITOR:** The main window displays the C code for Exercise8.c, which is identical to the code provided in the previous block.
- TERMINAL:** The bottom panel shows the output of running the program. It displays the prompt "Enter the string: ", the user input "bba", and the output "String Accepted by NFA".

The terminal output is as follows:

```
PS C:\TOCLAB> C:\TOCLAB\Exercise8.exe
Enter the string: bba
String Accepted by NFA
PS C:\TOCLAB> C:\TOCLAB\Exercise8.exe
Enter the string: 
```

Exercise 9

Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with 0 and end with 1

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len;

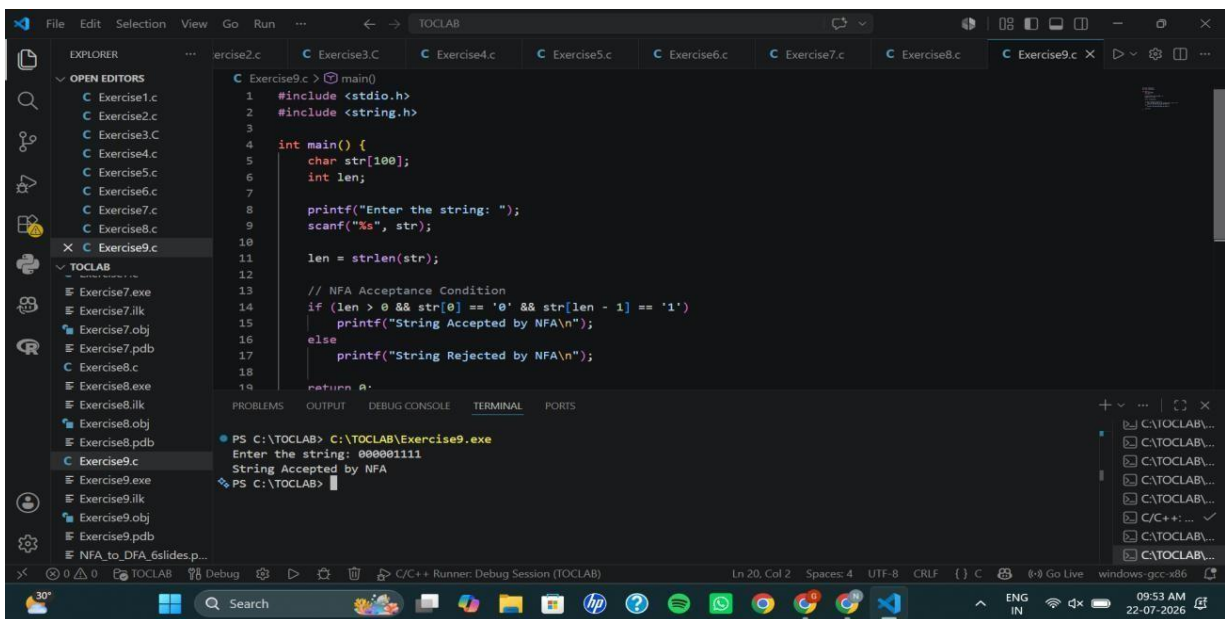
    printf("Enter the string: ");
    scanf("%s", str);

    len = strlen(str);

    // NFA Acceptance Condition
    if (len > 0 && str[0] == '0' && str[len - 1] == '1')
        printf("String Accepted by NFA\n");
    else
        printf("String Rejected by NFA\n");

    return 0;
}
```

Output:



```
File Edit Selection View Go Run ... TOCLAB
EXPLORER
  OPEN EDITORS
    Exercise1.c
    Exercise2.c
    Exercise3.c
    Exercise4.c
    Exercise5.c
    Exercise6.c
    Exercise7.c
    Exercise8.c
    Exercise9.c
  TOCLAB
    Exercise7.exe
    Exercise7.ilk
    Exercise7.obj
    Exercise7.pdb
    Exercise8.c
    Exercise8.exe
    Exercise8.ilk
    Exercise8.obj
    Exercise8.pdb
    Exercise9.c
    Exercise9.exe
    Exercise9.ilk
    Exercise9.obj
    Exercise9.pdb
    NFA_to_DFA_6slides.p...
  PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
    PS C:\TOCLAB> C:\TOCLAB\Exercise9.exe
    Enter the string: 000001111
    String Accepted by NFA
    PS C:\TOCLAB>
```

Exercise 10

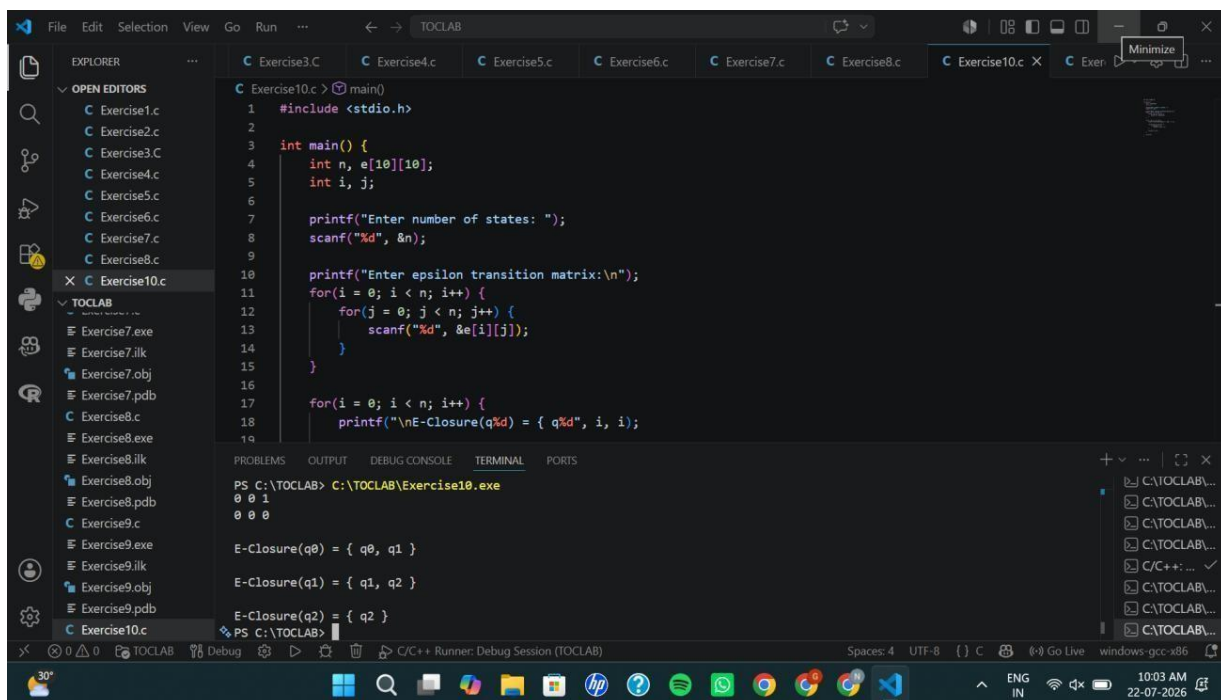
Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

Code:

```
#include <stdio.h>

int main() {
    int n, e[10][10];
    int i, j;
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon transition matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &e[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        printf("\nE-Closure(q%d) = { q%d", i, i);
        for(j = 0; j < n; j++) {
            if(e[i][j] == 1)
                printf(", q%d", j);
        }
        printf(" }\n");
    }
    return 0;
}
```

Output:



The screenshot shows a Visual Studio Code editor with a C program for Exercise 10. The program calculates the epsilon-closure for a Non-Deterministic Finite Automata (NFA) with 10 states. The output in the terminal shows the epsilon-closure for states q0, q1, and q2.

```
PS C:\TOCLAB> C:\TOCLAB\Exercise10.exe
0 0 1
0 0 0

E-Closure(q0) = { q0, q1 }
E-Closure(q1) = { q1, q2 }
E-Closure(q2) = { q2 }
```

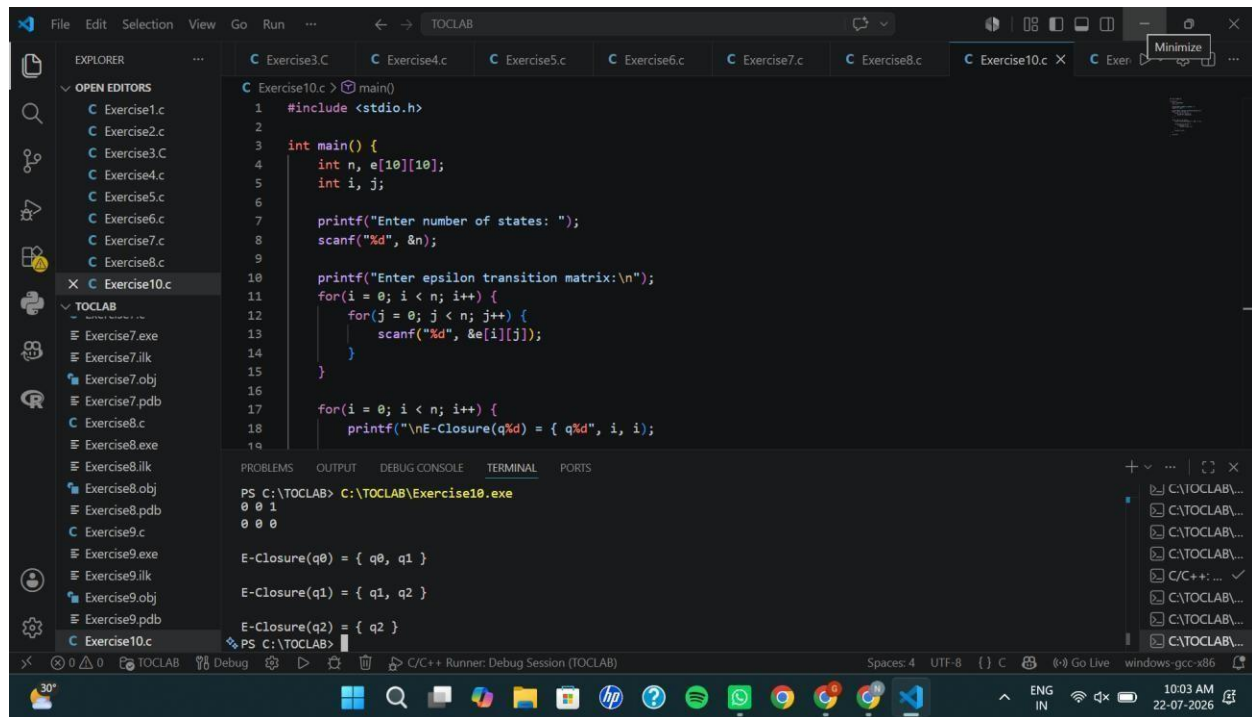
Exercise 11:

Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

Code:

```
#include <stdio.h>
int main() {
    int n, e[10][10];
    int i, j;
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon transition matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &e[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        printf("\nE-Closure(q%d) = { q%d", i, i);
        for(j = 0; j < n; j++) {
            if(e[i][j] == 1)
                printf(", q%d", j);
        }
        printf(" }\n");
    }
    return 0;
}
```

Output

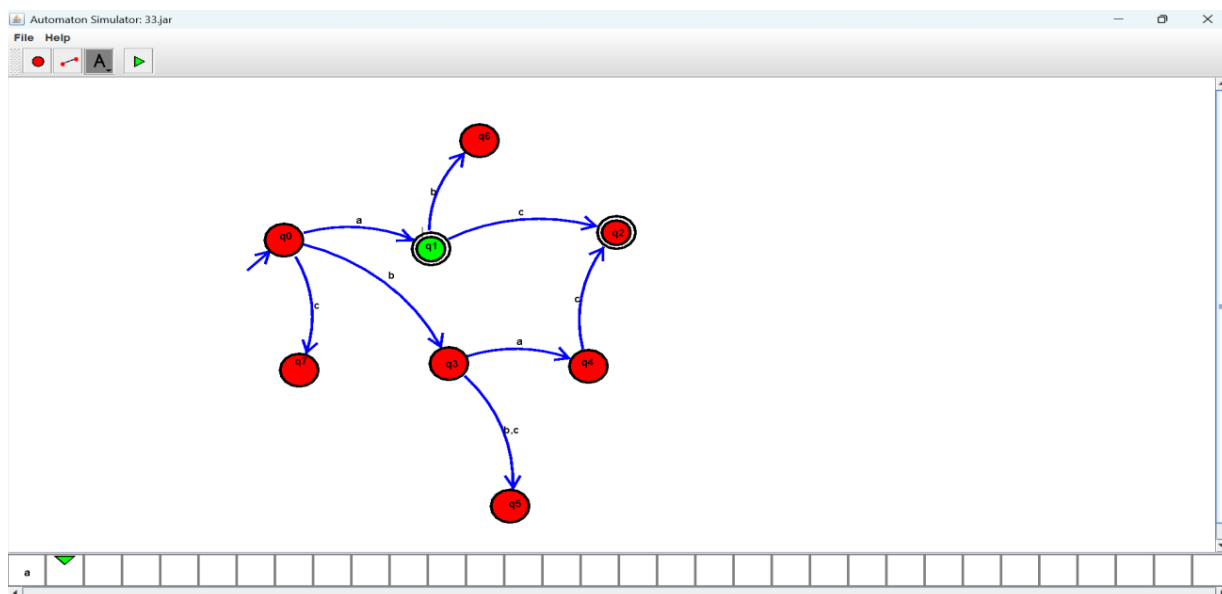


```
1 #include <stdio.h>
2
3 int main() {
4     int n, e[10][10];
5     int i, j;
6
7     printf("Enter number of states: ");
8     scanf("%d", &n);
9
10    printf("Enter epsilon transition matrix:\n");
11    for(i = 0; i < n; i++) {
12        for(j = 0; j < n; j++) {
13            scanf("%d", &e[i][j]);
14        }
15    }
16
17    for(i = 0; i < n; i++) {
18        printf("\nE-Closure(q%d) = { q%d", i, i);
19    }
```

PS C:\TOCLAB> C:\TOCLAB\Exercise10.exe
0 0 1
0 0 0
E-Closure(q0) = { q0, q1 }
E-Closure(q1) = { q1, q2 }
E-Closure(q2) = { q2 }

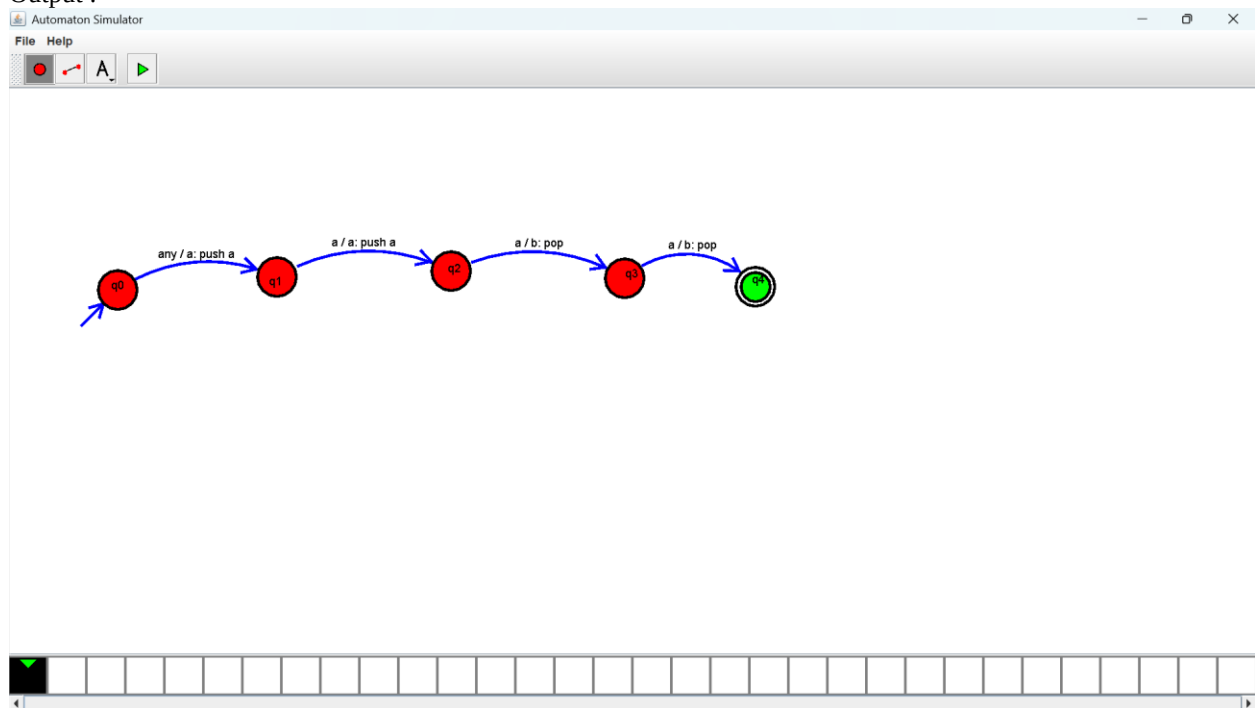
Exercise 12:

Output :



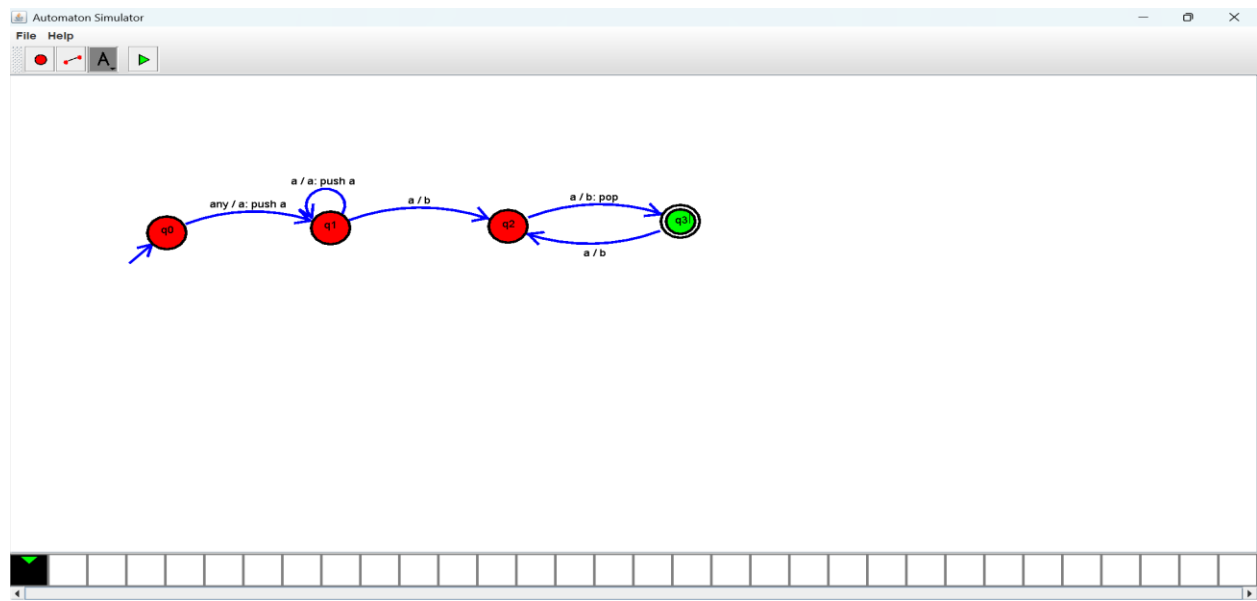
Exercise 13:

Output :



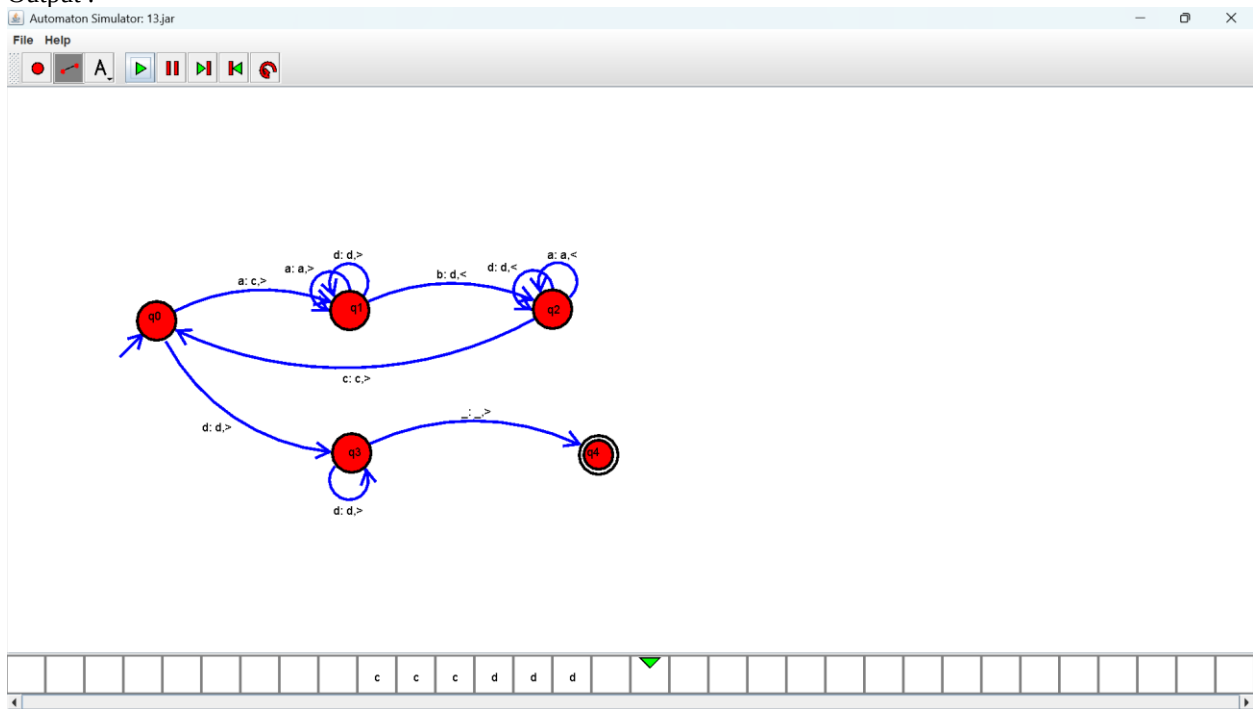
Exercise 14:

Output :



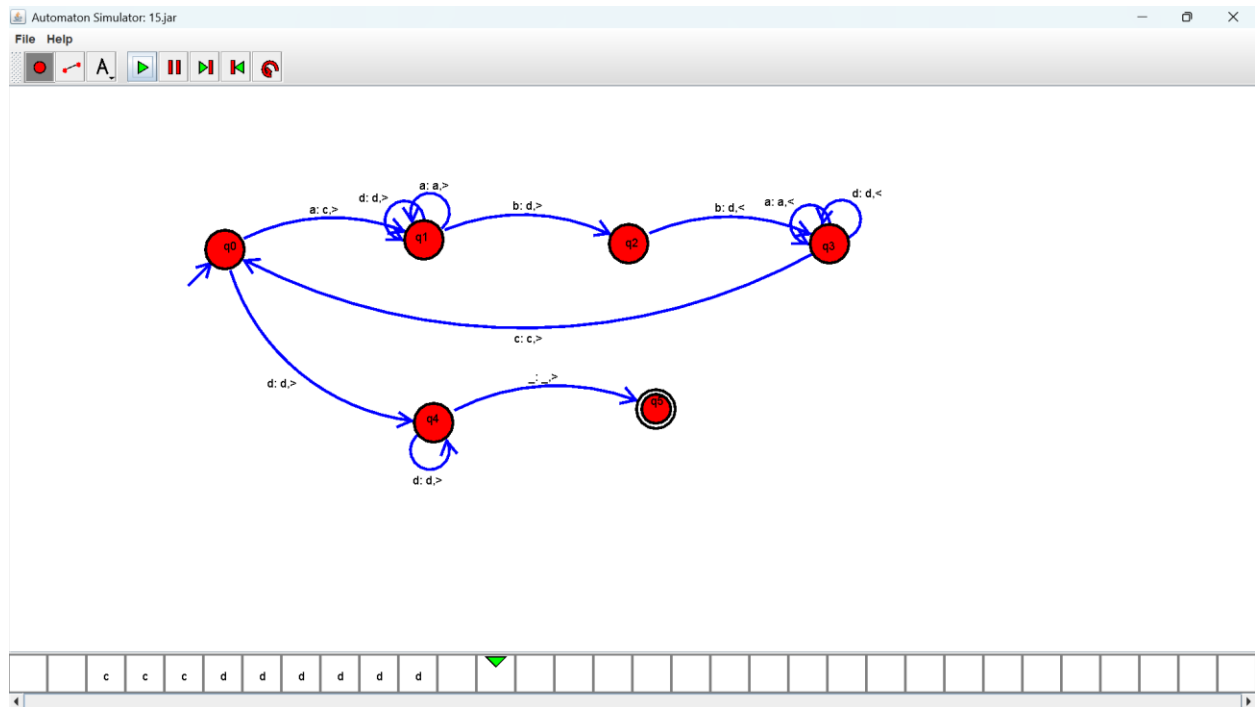
Exercise 15:

Output :



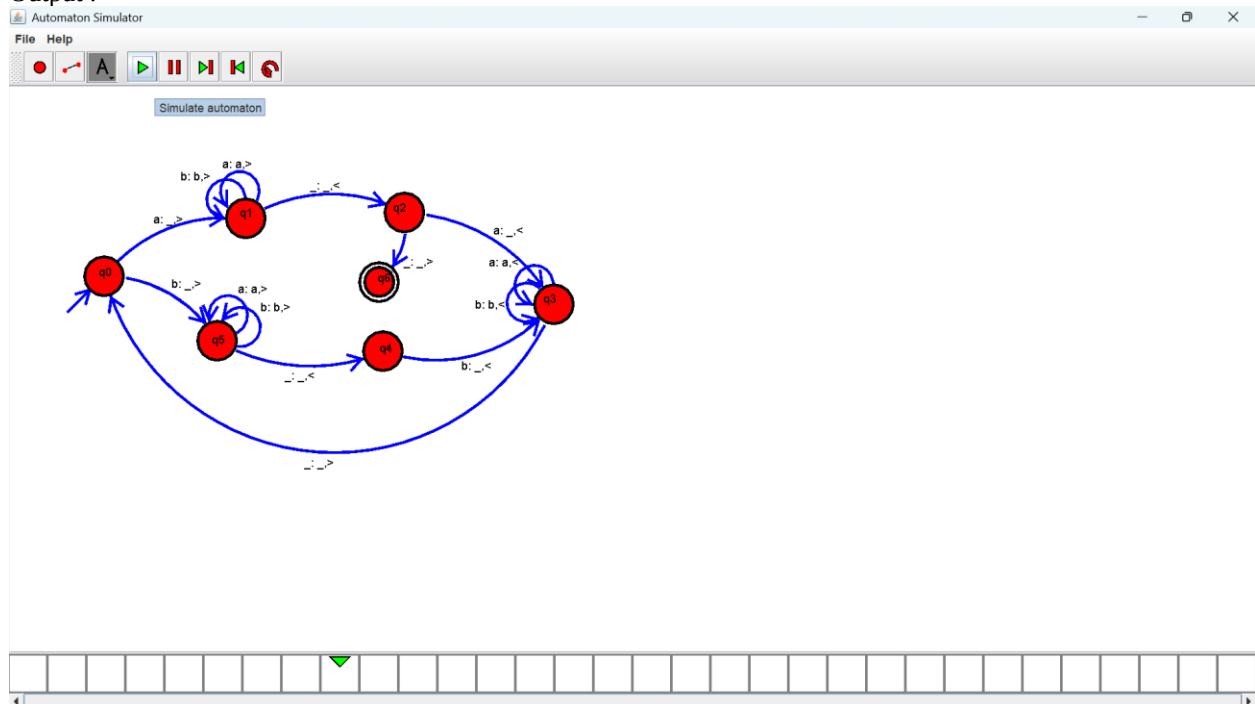
Exercise 16:

Output :



Exercise 17:

Output :

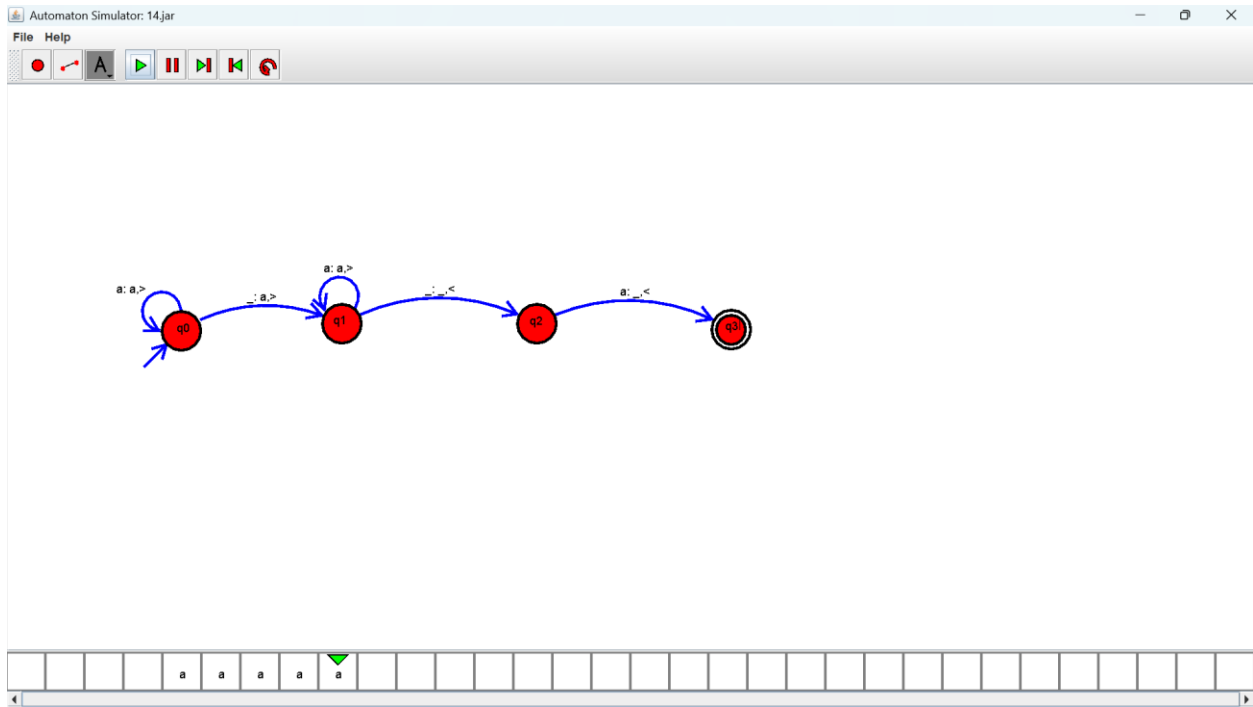


Exercise 18:

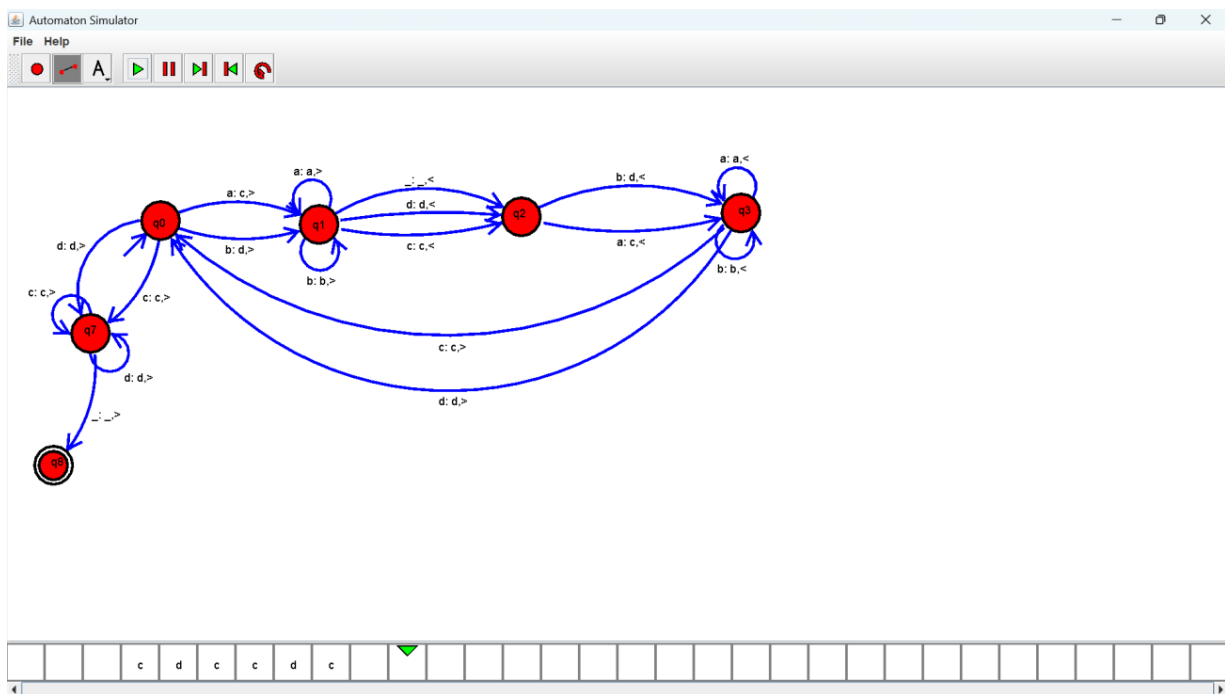
Output :

Exercise 19:

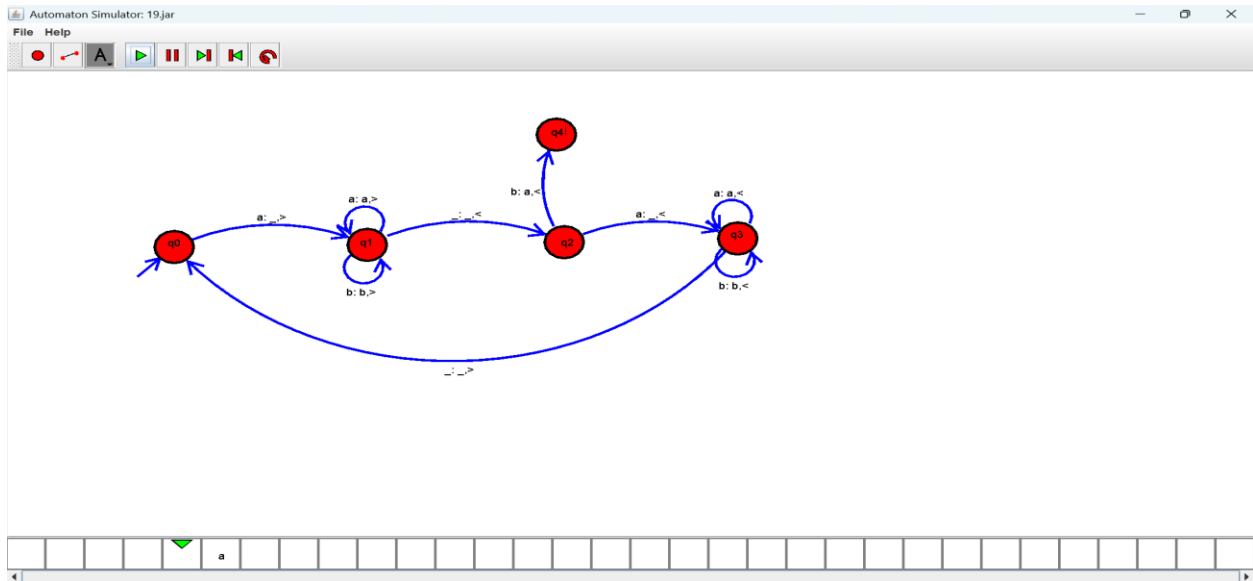
Output :



Exercise 20:

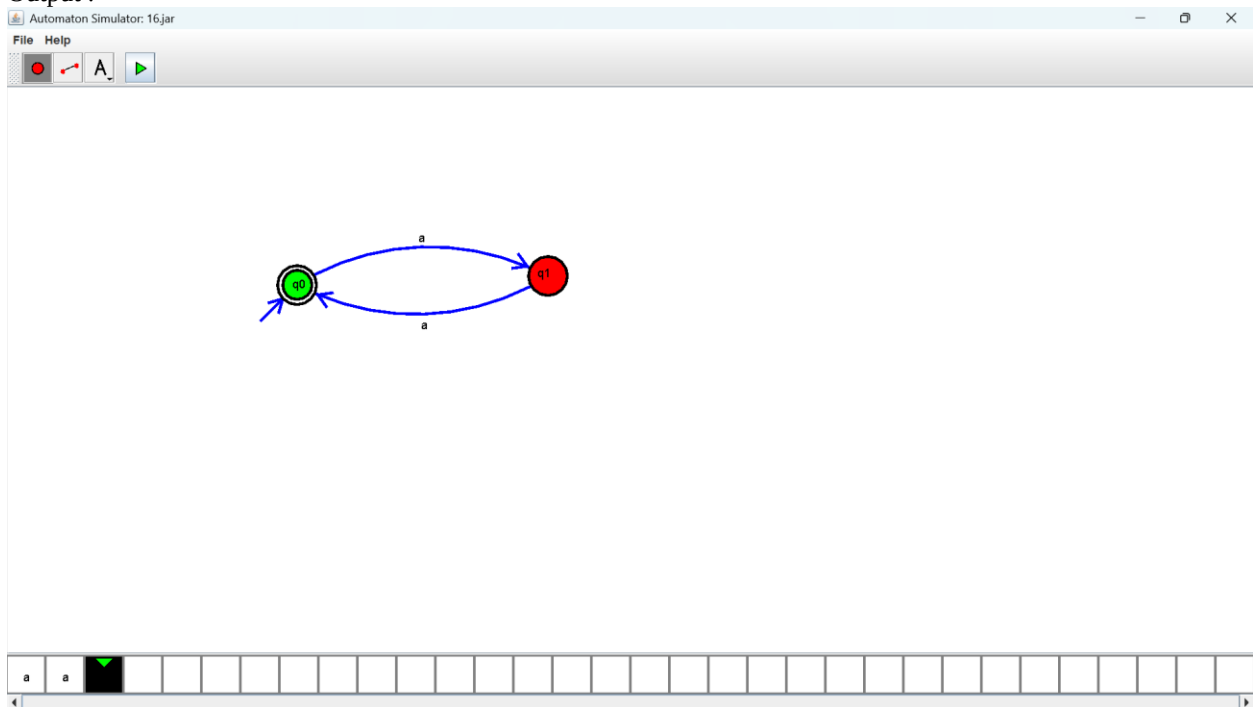


Output :



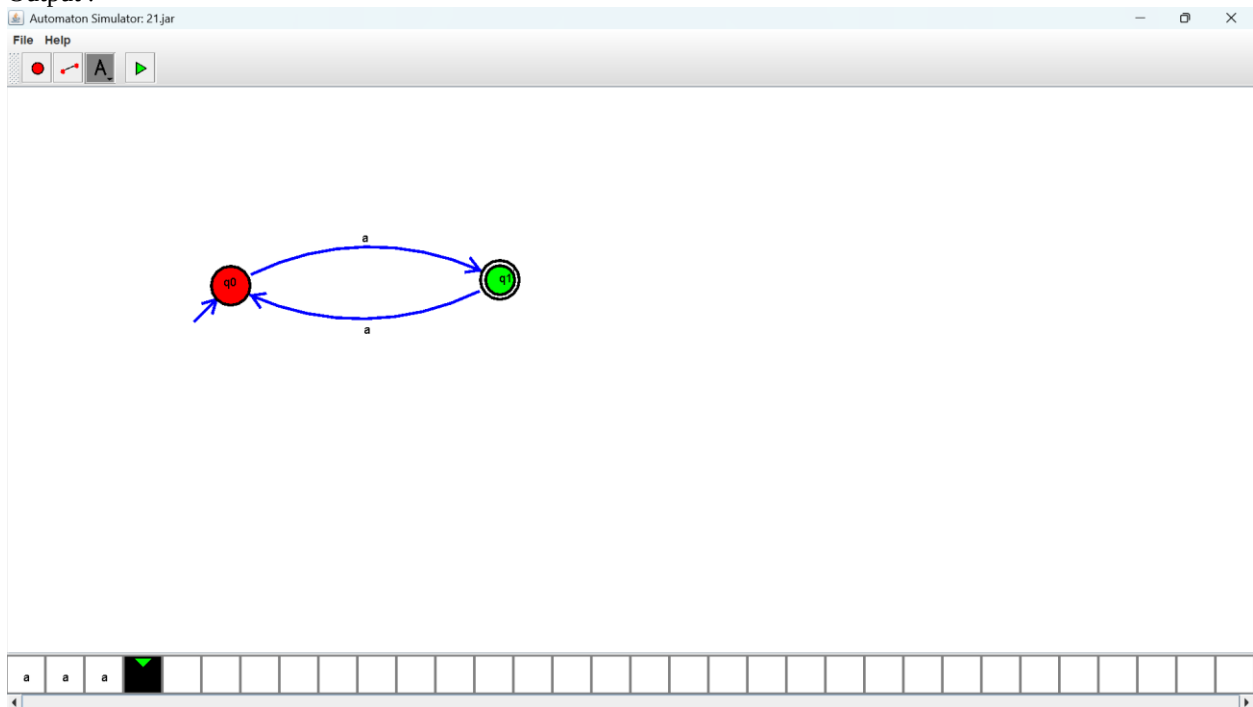
Exercise 21:

Output :



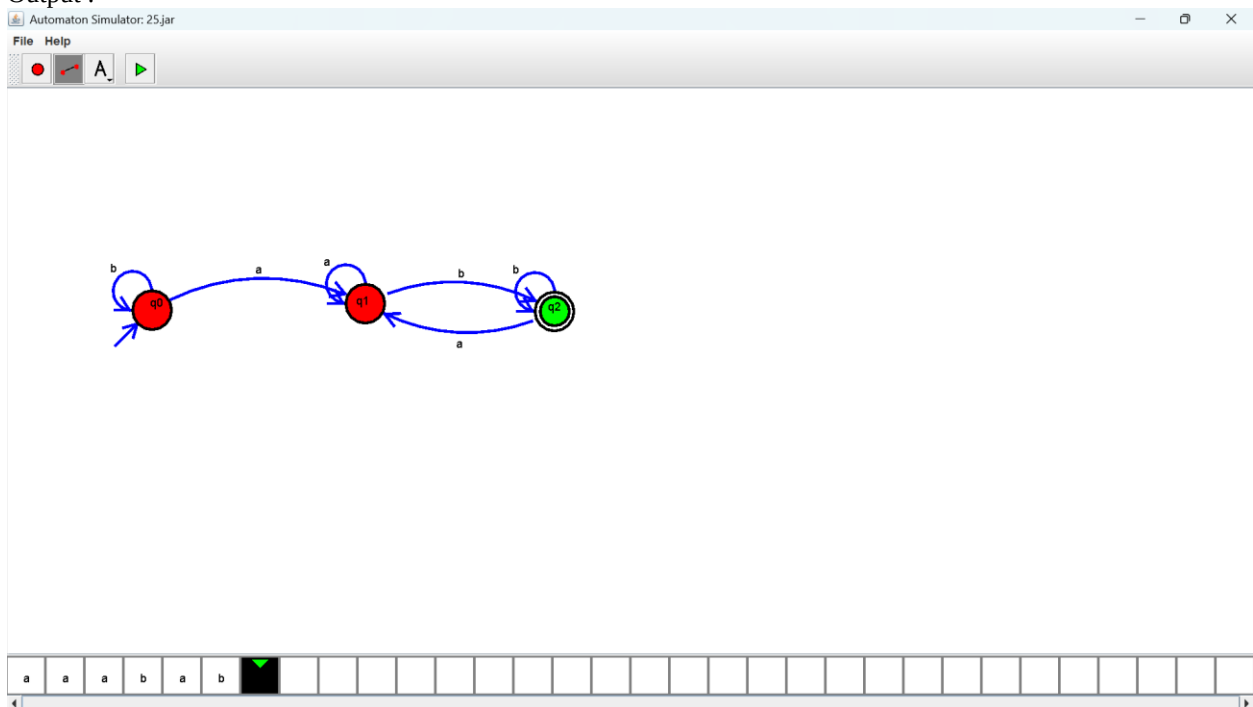
Exercise 22:

Output :



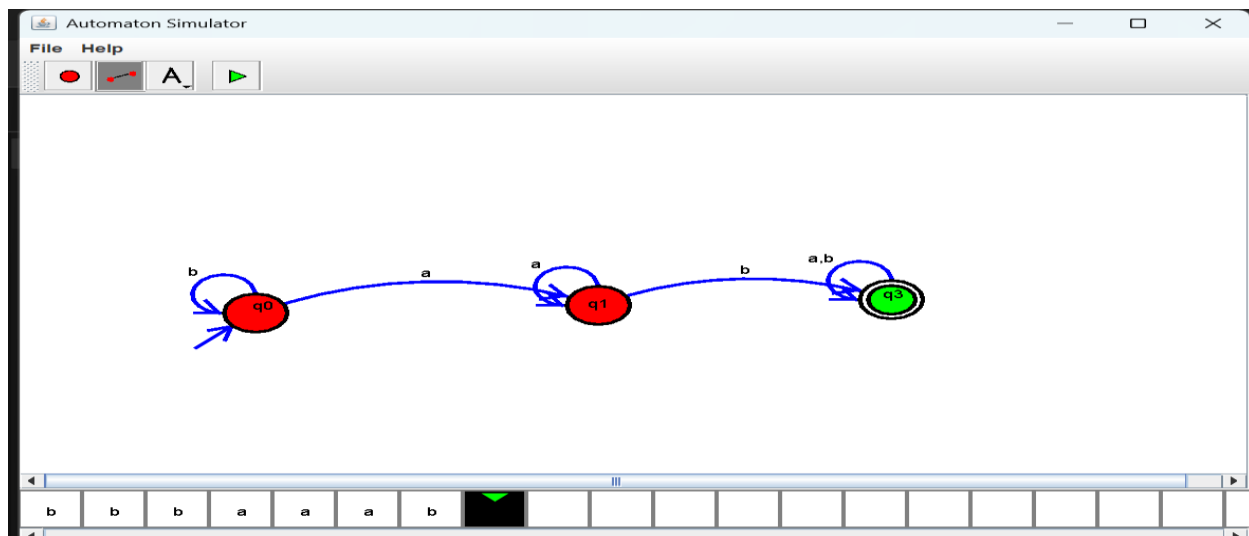
Exercise 23:

Output :



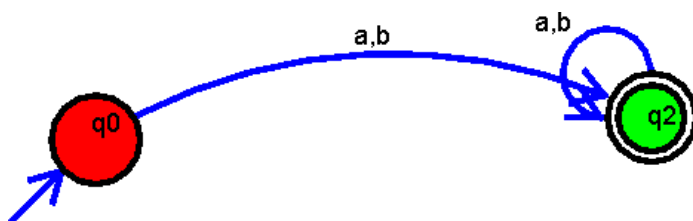
Exercise 24:

Output :



Exercise 25:

Output :



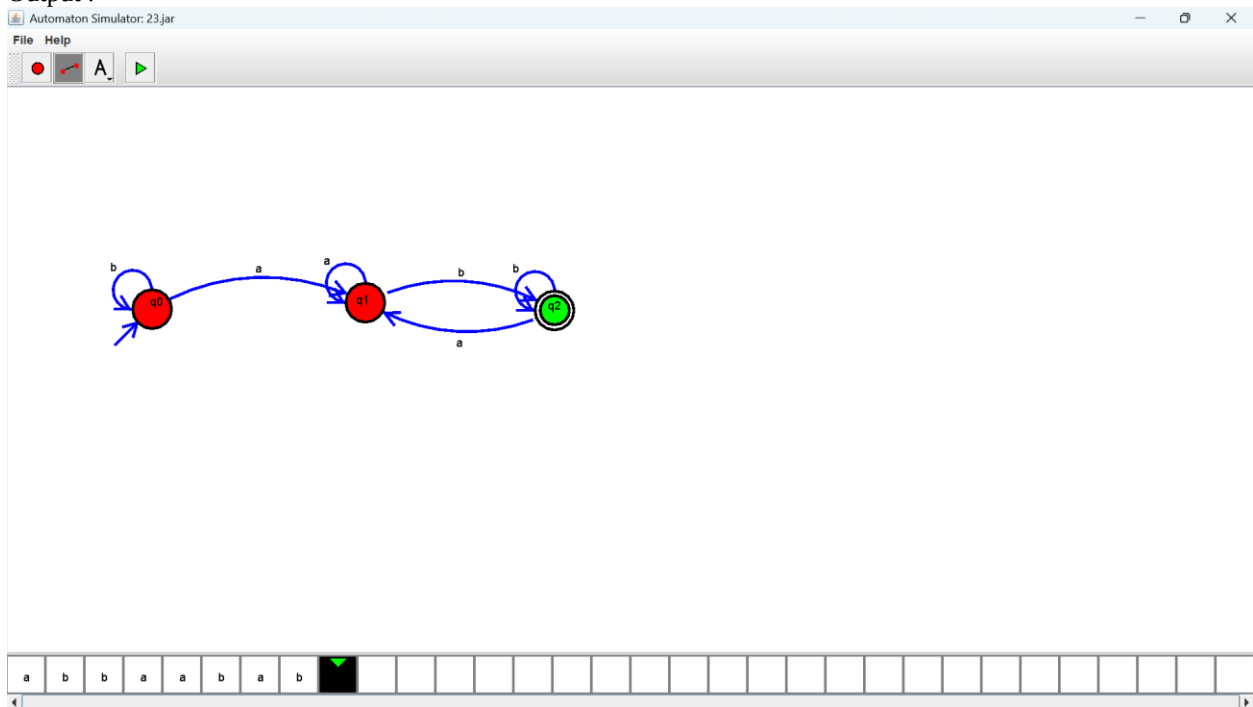
Exercise 26:

Output :

The screenshot shows the Automaton Simulator interface. The top toolbar includes icons for file operations (File, Help), a red circle, a green arrow, a red arrow, a green arrow, a red arrow, a green arrow, and a red arrow. The main window displays a finite state automaton with 11 states (red circles) and a start state (double circle). Transitions are labeled with input/output pairs (e.g., 'a: c:>', 'b: b:>'). The bottom status bar shows the sequence of inputs: 'c', 'd', 'c', 'c', 'd', 'c', and a green arrow.

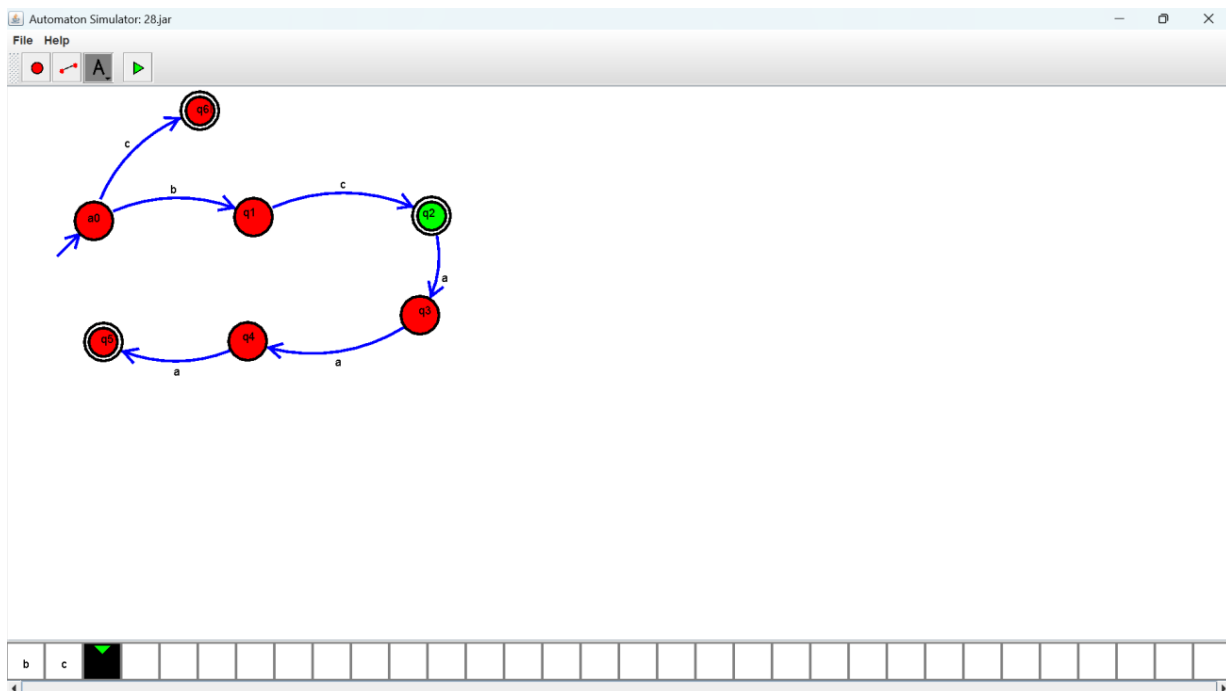
Exercise 28:

Output :



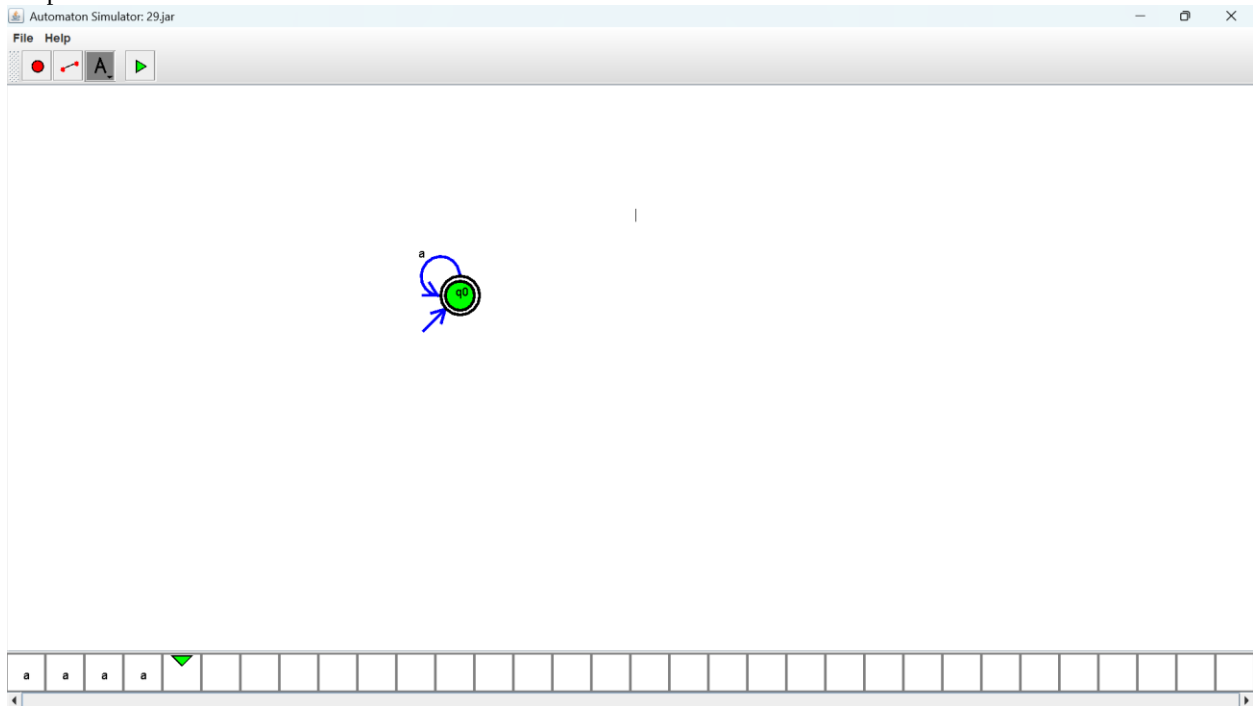
Exercise 29:

Output :



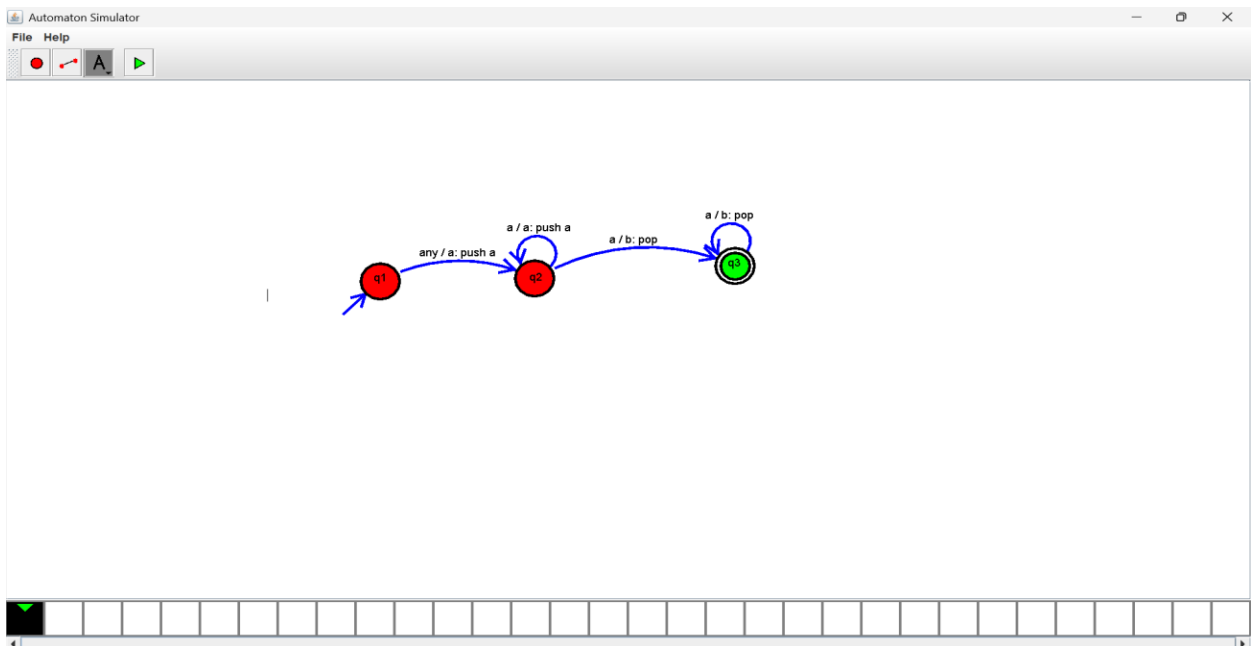
Exercise 30:

Output :



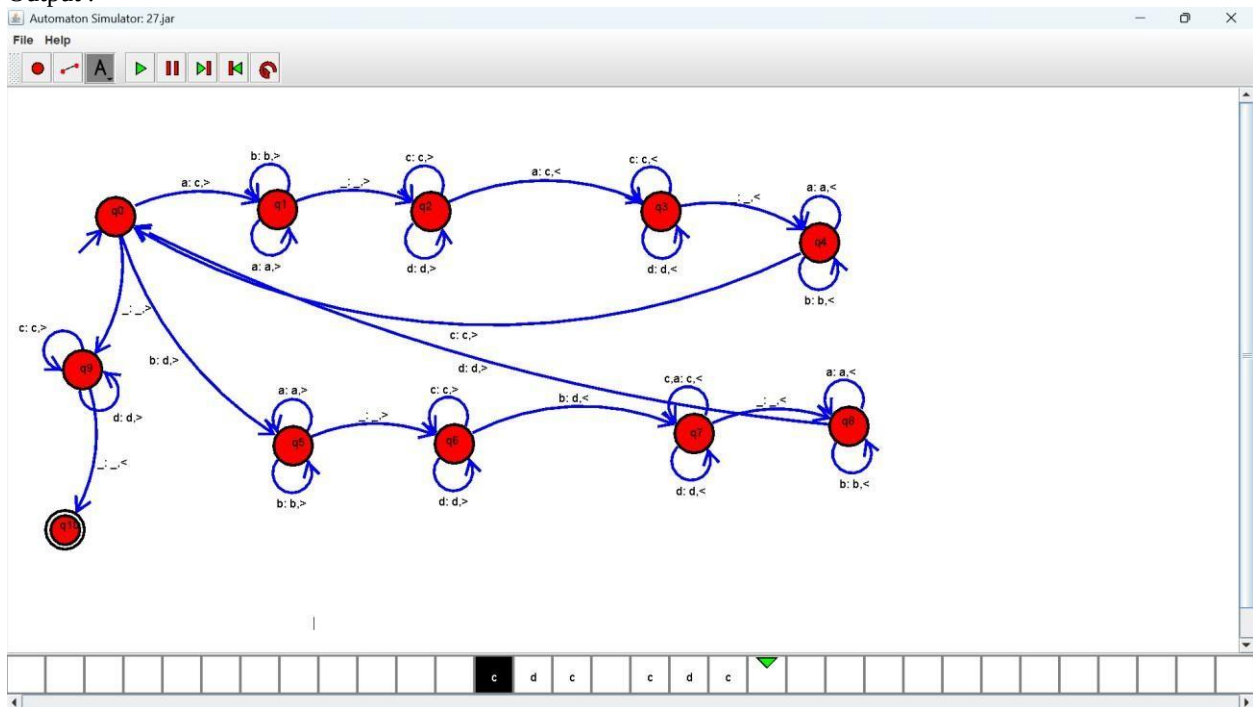
Exercise 31:

Output :



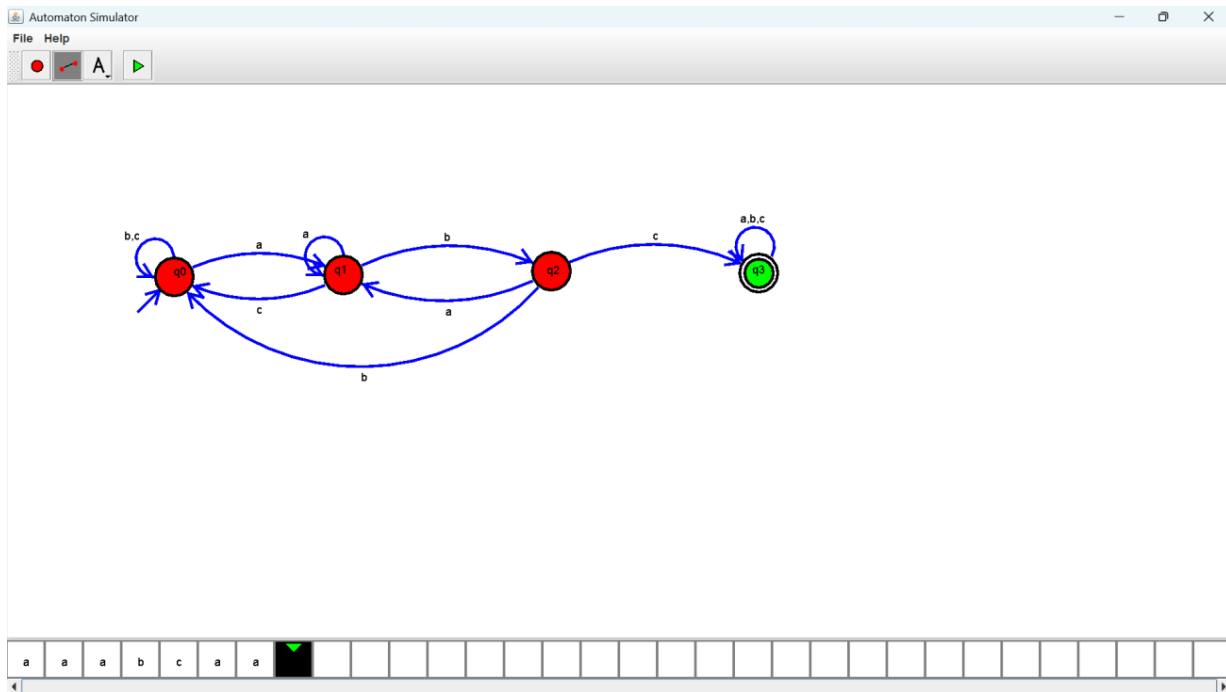
Exercise 32:

Output :



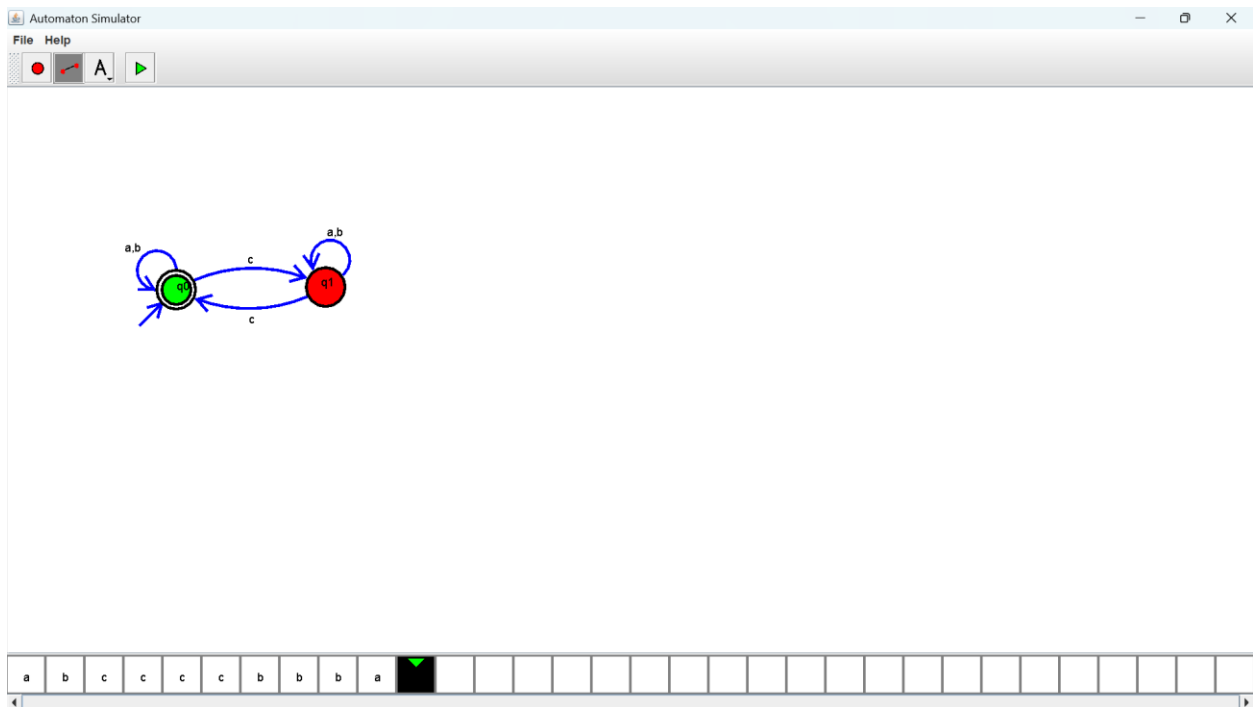
Exercise 33:

Output :



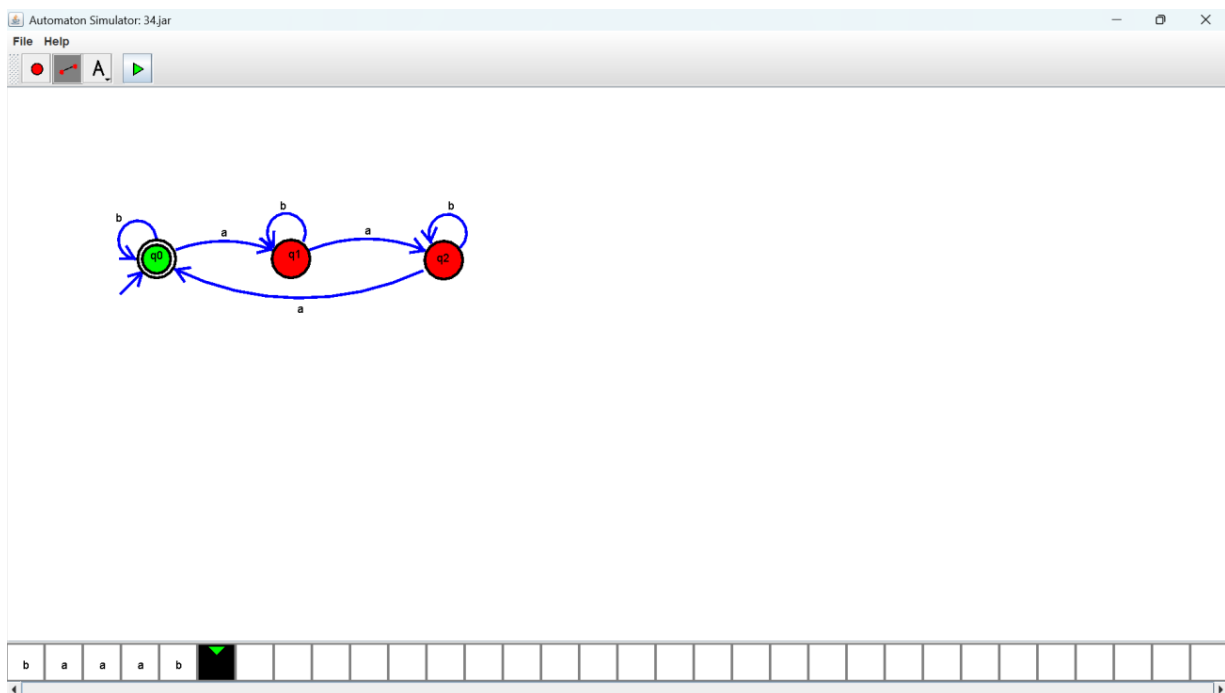
Exercise 34:

Output :



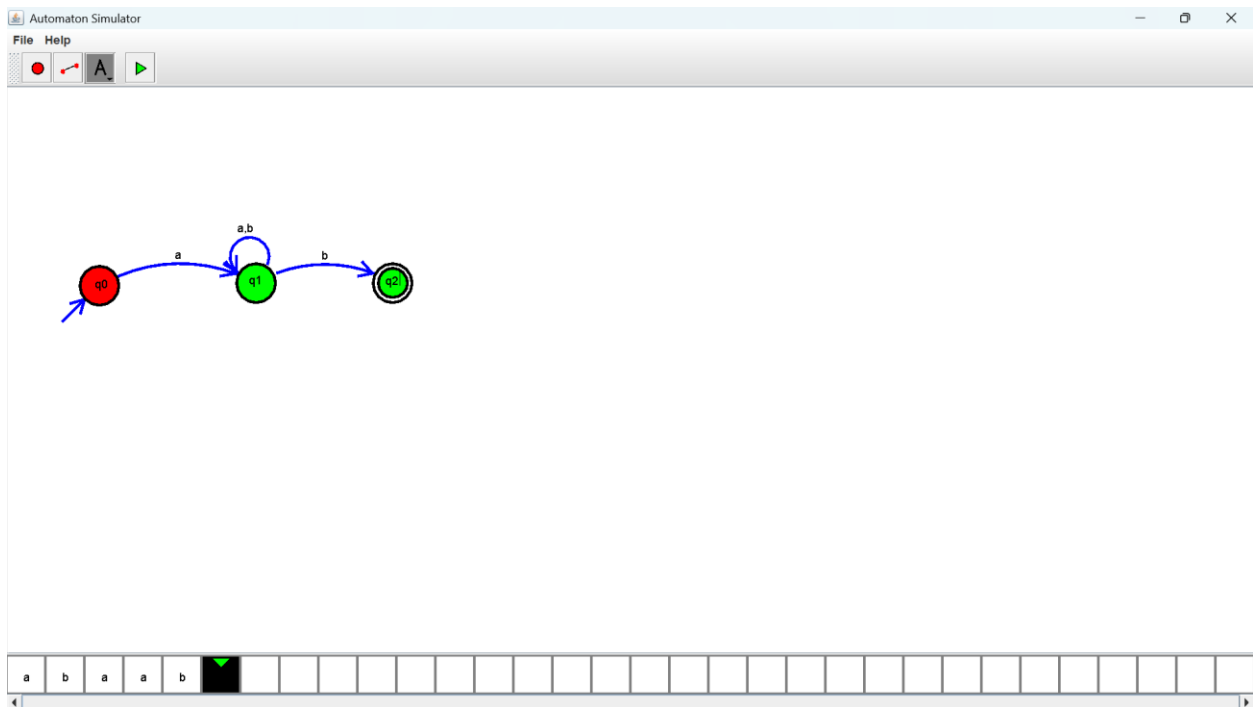
Exercise 35:

Output :



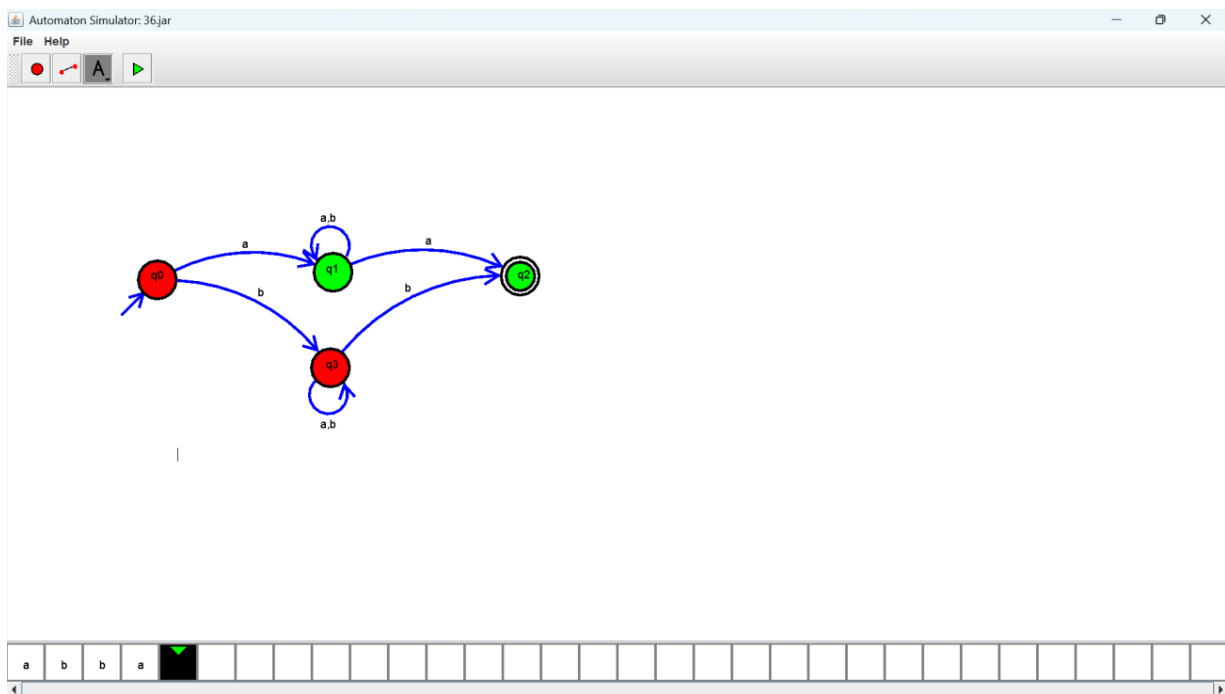
Exercise 36:

Output :



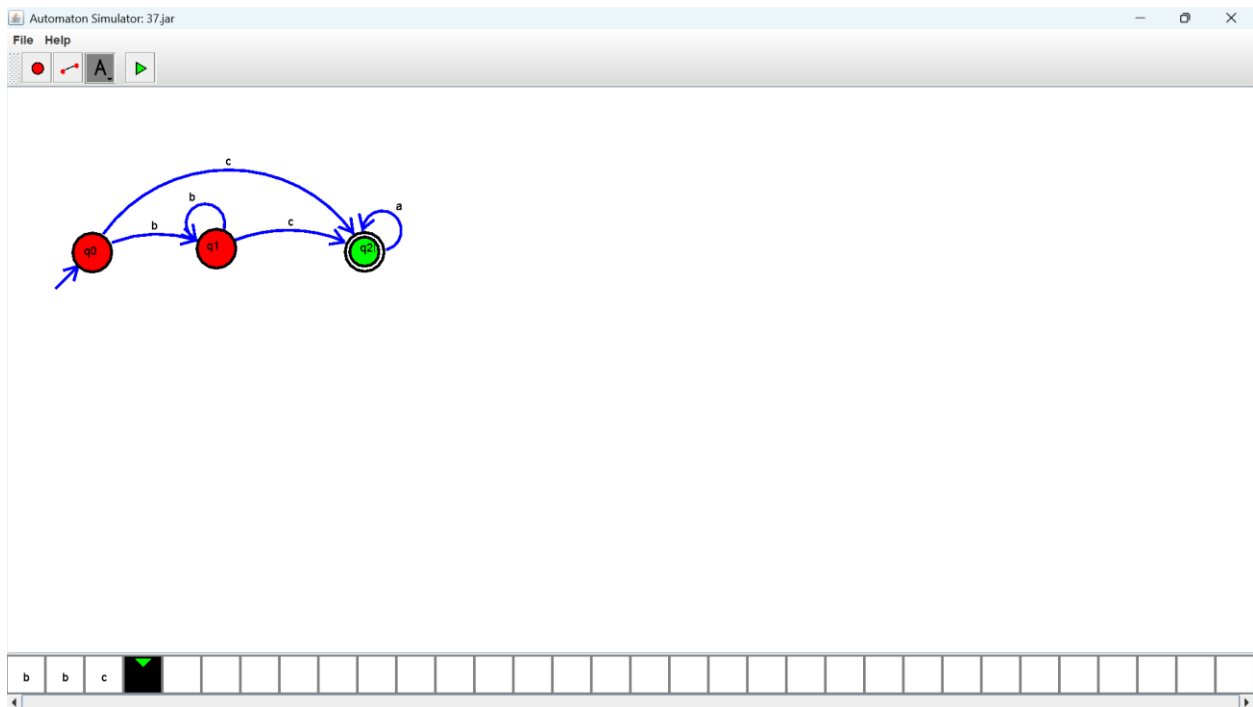
Exercise 37:

Output :



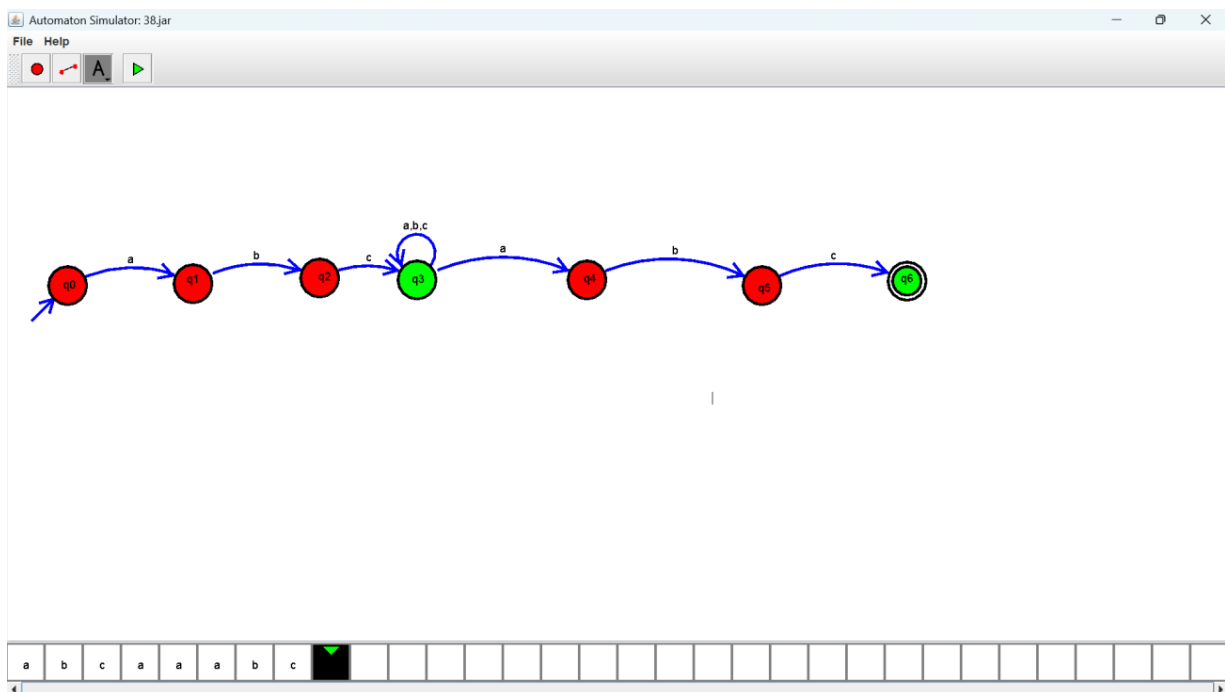
Exercise 38:

Output :



Exercise 39:

Output :



Exercise 40:

Output :

